



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:

Sistemas Operativos

Asignación:

Tarea #3: Incidente OOM Killer, informe RCA del servicio en el puerto 5001

Profesor:

Prof. Isacio Tamayo

Estudiante:

James Tack

Grupo:

1IL143

Fecha de entrega:

23 de septiembre de 2026

2do Semestre 2026

De qué se trata la tarea

La tarea plantea un incidente de producción. Acme Data Services tiene un microservicio Flask en el puerto 5001 que se cae a ratos, y cuando alguien revisa el contenedor lo encuentra detenido con código de salida 137. Me toca hacer de SRE, es decir, confirmar el síntoma, juntar evidencia, explicar la causa, mitigar y dejar una solución. La regla que manda sobre todo lo demás es que app.py no se toca. El arreglo tiene que salir del contenedor y del sistema operativo.

Trabajé en la máquina virtual del curso (vm-sistemas-operativos, Ubuntu 18.04 con kernel 4.15, 2 CPU y 2 GB de RAM), que armé con Vagrant en el laboratorio 1. Docker 24.0.2 ya estaba instalado. Todo lo que aparece en las figuras son capturas reales de las ventanas de terminal, tomadas en el momento en que corrió cada comando.

Campo	Dato
Identificación del incidente	INC-OOM-2026-001
Servicio afectado	API Flask reto1_app, puerto 5001 (5000 dentro del contenedor)
Detección / resolución	2026-09-23 14:09 / 2026-09-23 14:22
MTTR	0 h 13 min
Estado final / responsable	Resuelto a nivel de infraestructura / James Tack

Conceptos que usa el informe

El OOM Killer es la parte del kernel que entra en acción cuando un grupo de procesos se queda sin memoria y no hay forma de liberar páginas. Elige una víctima y le manda SIGKILL, una señal que no se puede atrapar ni ignorar. Por convención, un proceso que muere por una señal devuelve 128 más el número de la señal, y como SIGKILL es la 9, el contenedor queda con código 137. Los otros códigos que menciona la guía siguen la misma regla o son salidas normales. El 1 es una excepción no controlada, 130 es SIGINT (128 + 2) y 143 es SIGTERM (128 + 15).

Docker pone los límites de memoria por medio de los cgroups del kernel. En cgroups v2 cada contenedor tiene una carpeta con archivos que se leen y se escriben como texto. memory.max es el tope de memoria, memory.swap.max el tope de swap, memory.current el consumo del momento y memory.events lleva la cuenta de cuántas veces se tocó el límite (max), cuántas veces hubo OOM (oom) y cuántos procesos mató el kernel por eso (oom_kill). Desde /proc/<PID>/status se ve la memoria de un proceso. VmSize es todo el espacio virtual reservado, que incluye regiones que nunca se tocaron. VmRSS es lo que de verdad está en RAM, y VmSwap lo que se mandó a swap. Para decidir a quién matar el kernel calcula un puntaje, oom_score, que crece con la memoria del proceso y se corrige con oom_score_adj (el antiguo oom_adj), un valor entre -1000 y 1000 que el administrador puede ajustar.

Preparación del entorno

Paso 1. Encender la máquina virtual

La VM estaba guardada desde la tarea anterior y volvió con vagrant up sin problemas, así que no hubo que reinstalar nada. Más adelante tuve que reiniciarla y el arranque se quedó detenido en la carga de los discos, el mismo síntoma de la tarea 2 que viene de correr VirtualBox sobre Hyper-V en este equipo. Un VBoxManage controlvm reset lo resolvió.

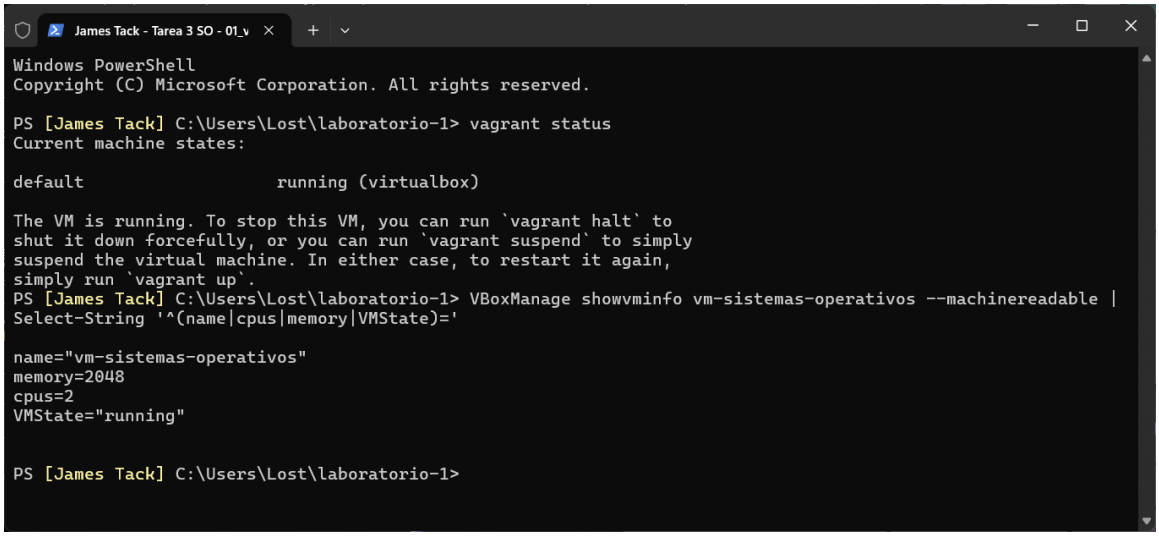


Figura 1. vagrant status y VBoxManage confirman que vm-sistemas-operativos está encendida con 2 CPU y 2048 MB.

Paso 2. Copiar los archivos a la VM

La tarea trae dos archivos, app.py y el Dockerfile. Los copié a ~/laboratorios/tarea3 con scp y la llave privada que genera Vagrant.



Figura 2. app.py y Dockerfile en Windows y copia a la VM con scp.

Paso 3. Pasar la VM a cgroups v2

Este paso no está en la guía, pero sin él no se podía seguir. Toda la guía habla de memory.max, memory.events y de la ruta /sys/fs/cgroup/system.slice/docker-<id>.scope, que son nombres de cgroups v2. Ubuntu 18.04 arranca con cgroups v1, y al revisar docker info salían dos líneas que lo confirmaban, Cgroup Version: 1 y WARNING: No swap limit support. En v1 esos archivos se llaman distinto (memory.limit_in_bytes, memory.oom_control) y, sin contabilidad de swap, Docker ignora --memory-swap.

Agregué dos parámetros al arranque del kernel en un archivo aparte, /etc/default/grub.d/99-cgroupv2.cfg, para no tocar la configuración original. systemd.unified_cgroup_hierarchy=1 hace que systemd monte solo la jerarquía v2, y swapaccount=1 activa la contabilidad de swap. Después corrí update-grub y reinicié. Para volver atrás basta con borrar ese archivo y repetir update-grub.

```
GRUB_CMDLINE_LINUX="$GRUB_CMDLINE_LINUX systemd.unified_cgroup_hierarchy=1 swapaccount=1"
sudo update-grub && sudo reboot
```

Por suerte, el kernel 4.15 ya trae los controladores de memoria y CPU para v2, así que Docker los tomó sin quejarse. Además, con el driver systemd las rutas quedaron idénticas a las de la guía. Aproveché para poner la zona horaria de Panamá en la VM, porque las horas del timeline tienen que coincidir entre dmesg, docker y mis comandos.

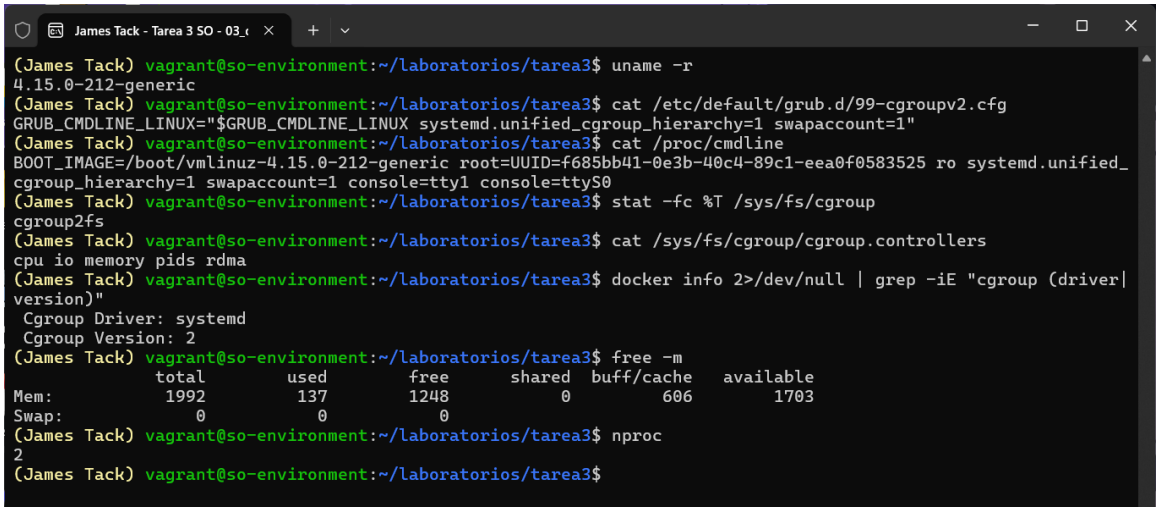


Figura 3. El kernel arranca con los nuevos parámetros, /sys/fs/cgroup es cgroup2fs y Docker reporta Cgroup Version: 2. La VM no tiene swap.

Paso 4. Revisar el Dockerfile y app.py

La imagen parte de python:3.11-slim, instala Flask y arranca python app.py, que queda como PID 1 del contenedor. No hay nada raro en el Dockerfile. En cambio, el problema está en app.py, y aunque no lo puedo modificar sí lo puedo leer para saber qué buscar.

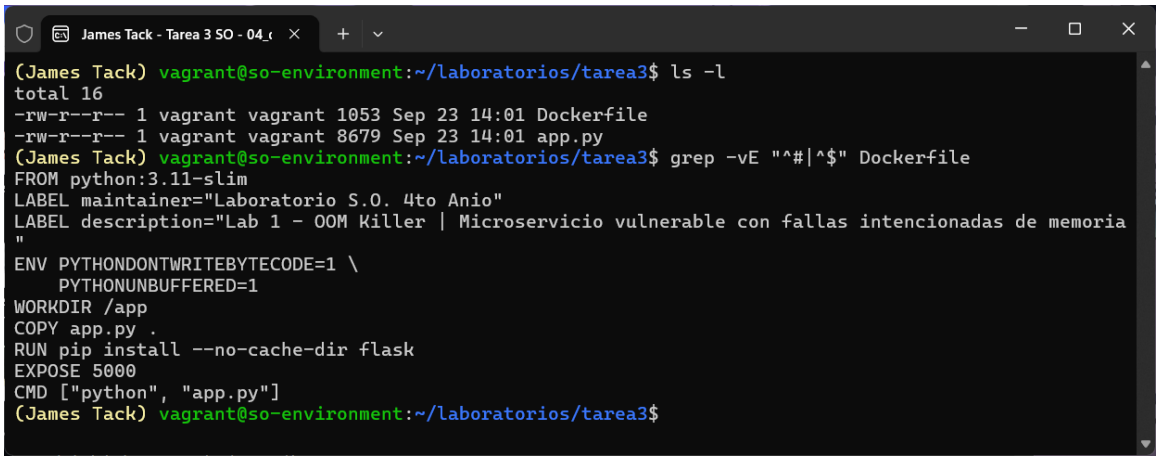


Figura 4. Dockerfile del microservicio sin comentarios.

La ruta /api/v1/process-data crea un bytearray del tamaño pedido, escribe un byte cada 4096 para que el kernel asigne de verdad cada página y lo agrega a LEAK_BUCKETS, una lista global que nadie vacía.

Cada llamada deja su bloque retenido para siempre. Esa es la fuga.

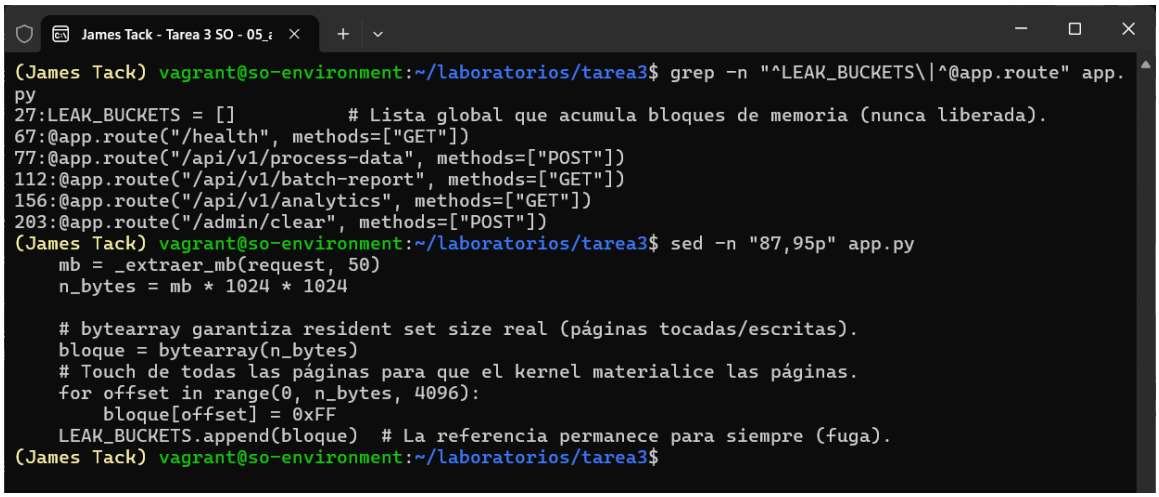


Figura 5. Rutas de app.py y el fragmento de /process-data que agrega cada bloque a LEAK_BUCKETS.

Paso 5. Construir la imagen

```
docker build -t app-reto1-oom:v1 .
```

Tardó unos 10 segundos, casi todo en pip install flask, y la imagen quedó de 130 MB.

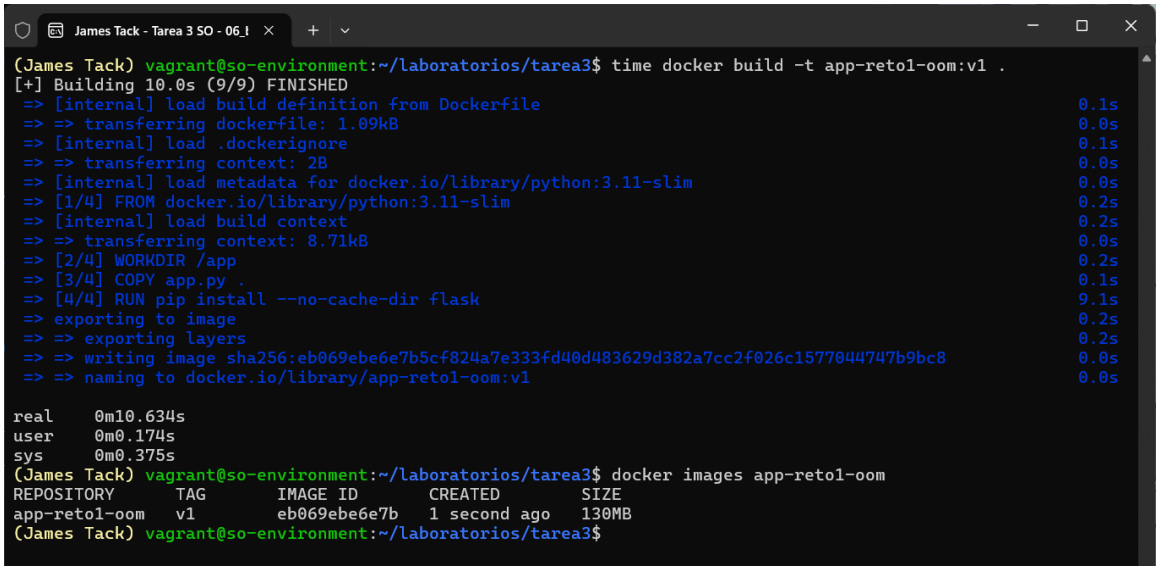


Figura 6. docker build termina y app-reto1-oom:v1 aparece en docker images.

Paso 6. Levantar el servicio con el límite estricto

Lo levanté con la configuración que trae la guía, 256 MB de RAM, --memory-swap igual a --memory (o sea, cero swap), una CPU y sin reinicio automático. /health respondió con pid 1.

```
docker run -d --name reto1_app --restart no -p 5001:5000 \
  --memory="256m" --memory-swap="256m" --cpus="1" app-reto1-oom:v1
```

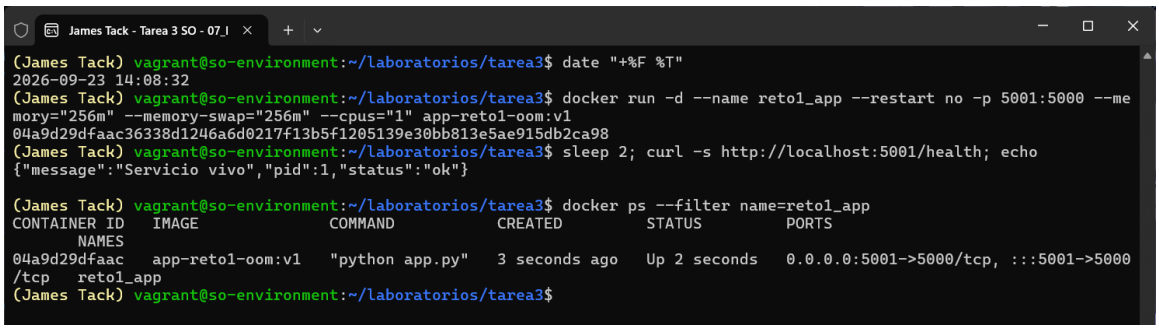


Figura 7. Contenedor reto1_app en ejecución y respuesta de /health a las 14:08.

La primera vez que lo levanté (14:05) la VM se congeló unos 40 segundos en medio de la prueba y la ventana de captura se cerró con una sola llamada hecha. Borré ese contenedor con docker rm -f y empecé de cero, como indica la guía. Todas las figuras siguientes son del contenedor 04a9d29dfaac.

Detección y diagnóstico

Paso 7. Demostrar dónde se ejecuta cada comando

La guía exige separar lo que se ve desde el host de lo que se ve dentro del contenedor, porque un cat /proc/1/status en el host lee el proceso 1 de la VM y no el de la aplicación. La figura 8 lo deja claro. Dentro del contenedor el hostname es 04a9d29dfaac, que son los primeros 12 caracteres del ID que

da docker ps, y el PID 1 se llama python y ejecuta app.py. Los mismos dos comandos en el host devuelven so-environment y systemd.

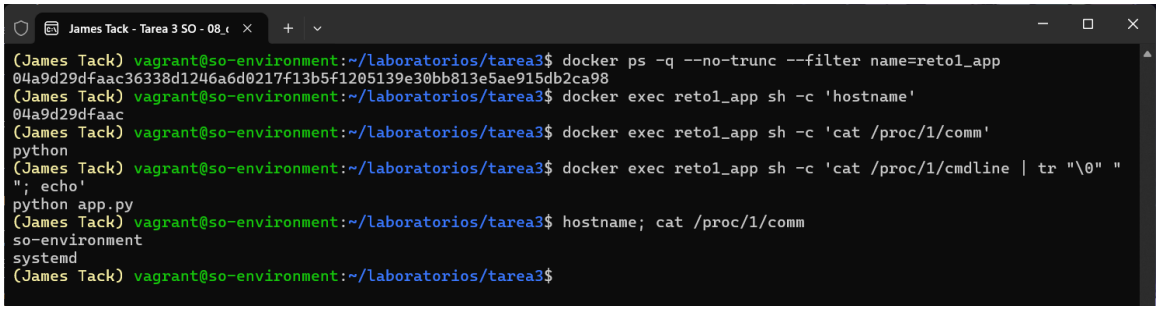


Figura 8. hostname y /proc/1/comm dentro del contenedor (docker exec) contra los mismos comandos en el host.

Dónde	Comandos	Qué se ve
Host (VM)	docker ps/inspect/logs/stats/events, dmesg, free, vmstat	El kernel real y la vista de Docker
Contenedor	docker exec reto1_app sh -c '...'	/proc y /sys/fs/cgroup del contenedor
Host (cgroup del contenedor)	cat/tee /sys/fs/cgroup/system.slice/docker-<id>.scope/...	Los mismos archivos del cgroup, vistos desde afuera

Paso 8. Límites y línea base

docker inspect confirma los 268435456 bytes (256 MiB) de Memory y MemorySwap. Dentro del contenedor, memory.max marca el mismo número y memory.swap.max vale 0, que es lo que describe la nota del SRE veterano. Cuando --memory-swap es igual a --memory, el swap disponible es la diferencia, cero. memory.events tiene todos los contadores en 0.

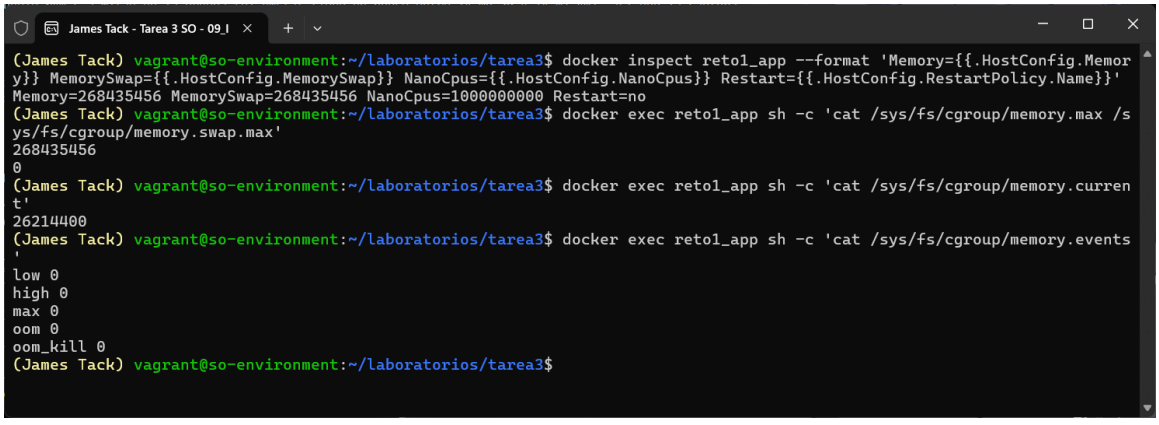


Figura 9. Límites vistos por Docker y por el cgroup del contenedor antes del incidente.

En reposo, python ocupa 34784 kB de RSS y reserva 116924 kB de espacio virtual. oom_score vale 17 y oom_score_adj 0, así que nadie tocó la prioridad del proceso ante el OOM Killer. docker stats marca 24,46 MiB sobre 256 MiB y la VM tiene más de 1,6 GB disponibles.

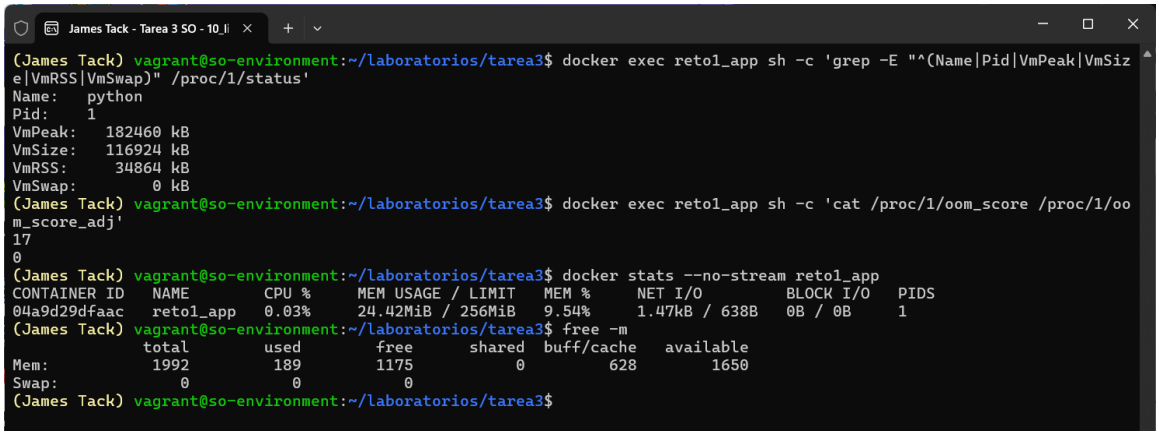


Figura 10. VmRSS, oom_score, docker stats y free -m antes de la fuga.

Paso 9. Reproducir el incidente

Dejé abierta una ventana con docker stats cada cinco segundos y en otra lancé las seis llamadas de 64 MB que pide la guía, con la hora de cada una y el VmRSS leído desde adentro del contenedor después de cada respuesta.

```
for i in $(seq 1 6); do
  curl -s -X POST "http://localhost:5001/api/v1/process-data?mb=64"
  docker exec reto1_app sh -c 'grep VmRSS /proc/1/status; cat /sys/fs/cgroup/memory.current'
  sleep 5
done
```

Las tres primeras llamadas respondieron bien y el RSS subió a 100, 166 y 231 MB, unos 65 MB por llamada. La cuarta, a las 14:09:49, no recibió respuesta. curl terminó con código 52, que significa que el servidor cerró la conexión sin contestar, y las dos siguientes dieron código 7 porque ya no había nadie escuchando en el puerto. docker ps mostró Exited (137).

El servicio aguantó tres llamadas. La cuarta ya no cabía.

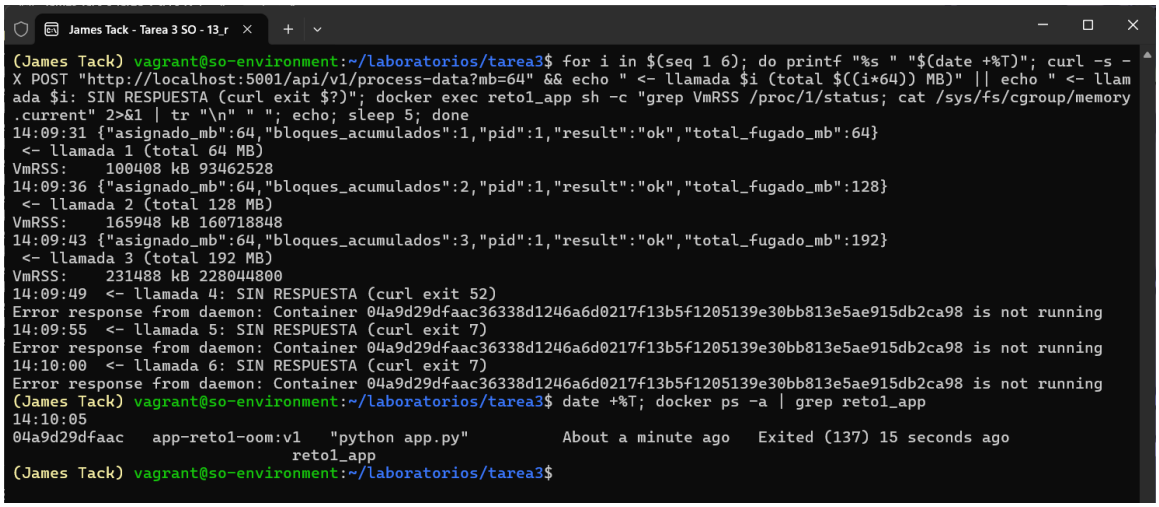


Figura 11. Seis llamadas de 64 MB. La cuarta mata el servicio y el contenedor queda en Exited (137).

La ventana de docker stats cuenta lo mismo desde el host. El uso pasa de 24 MiB a 152, luego a 216 MiB (84,7 %) y después cae a 0 B sobre 0 B, que es lo que Docker muestra cuando el contenedor ya no existe como proceso.

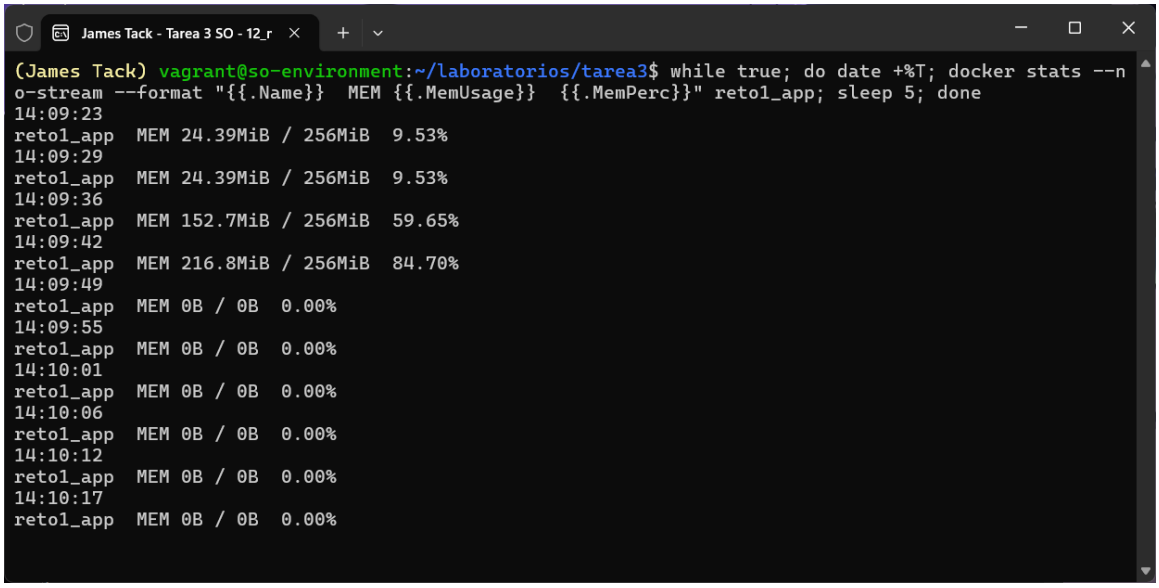


Figura 12. docker stats durante la reproducción.

Paso 10. Evidencia después de la caída

Antes de tocar nada junté lo que dejó el kernel. dmesg tiene la línea Killed process 5867 (python) con 259896 kB de anon-rss, y docker inspect confirma ExitCode=137 y OOMKilled=true. El PID 5867 es el número de python visto desde el host. Adentro del contenedor ese proceso es el PID 1.

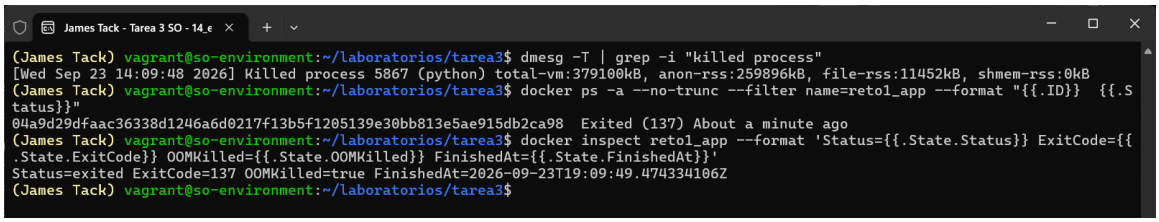


Figura 13. La firma del OOM Killer en dmesg y el estado del contenedor según Docker.

El informe completo del kernel dice más. La primera línea cuenta que fue python quien invocó al OOM Killer al pedir una página nueva, y la pila pasa por mem_cgroup_out_of_memory, la ruta del OOM de un cgroup y no la del sistema entero. La línea Task in /system.slice/docker-04a9d29dfaac....scope killed as a result of limit of el mismo scope señala al límite del contenedor como culpable. Más abajo aparecen memory: usage 262144kB, limit 262144kB, con el uso exactamente en el tope, y failcnt 17, las veces que el cgroup chocó con el límite antes de rendirse.

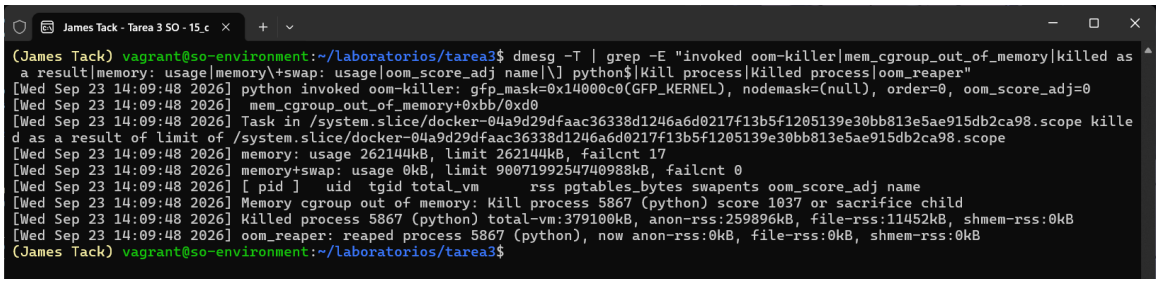


Figura 14. Líneas clave del informe del OOM Killer en dmesg.

El puntaje que usó el kernel fue 1037 y no 17 como en la línea base. En un OOM de cgroup el puntaje se calcula contra el límite del cgroup, 65536 páginas de 4 KB, y no contra la RAM de la VM. Las 67837 páginas de RSS más las tablas de páginas pasan un poco del 100 % de ese límite, y por eso el número supera 1000. oom_reaper liberó la memoria del proceso en el mismo segundo.

Además, docker logs no trae ninguna traza de Python. Aparecen el arranque de Flask, el /health y tres POST con código 200, y la cuarta petición ni siquiera quedó registrada. Sin embargo, eso también es evidencia. Si la aplicación hubiera fallado por su cuenta habría un traceback o un MemoryError, y el código sería 1.

Un SIGKILL no le da tiempo al proceso de escribir nada.

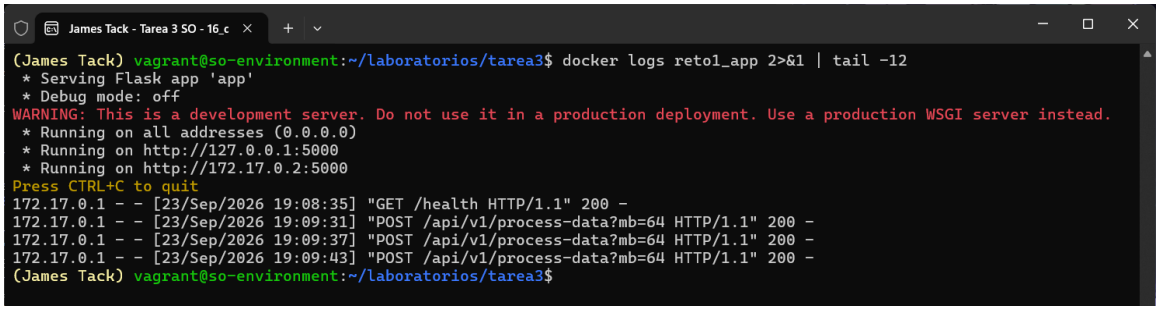


Figura 15. docker logs termina en la tercera llamada, sin error de la aplicación.

El contador oom_kill de memory.events no se puede leer después de la caída, porque el cgroup desaparece junto con el contenedor. Para tener esa misma señal registrada desde el host usé docker events, que guarda un evento oom justo antes del die con código 137.

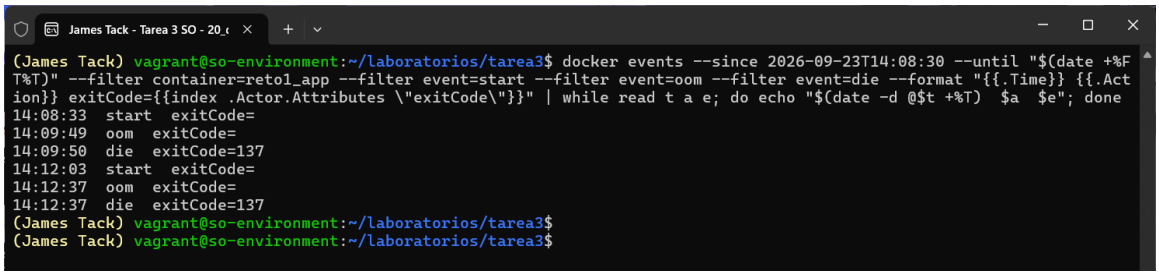


Figura 16. docker events registra oom y die 137 en las dos caídas.

Paso 11. Levantar de nuevo con la misma configuración

Con el servicio muerto, la única mitigación posible es arrancarlo otra vez. Usé docker start, que conserva el mismo contenedor y la misma configuración. /health volvió a responder, memory.max seguía en 256 MB y oom_kill arrancó en 0, porque el cgroup es nuevo.

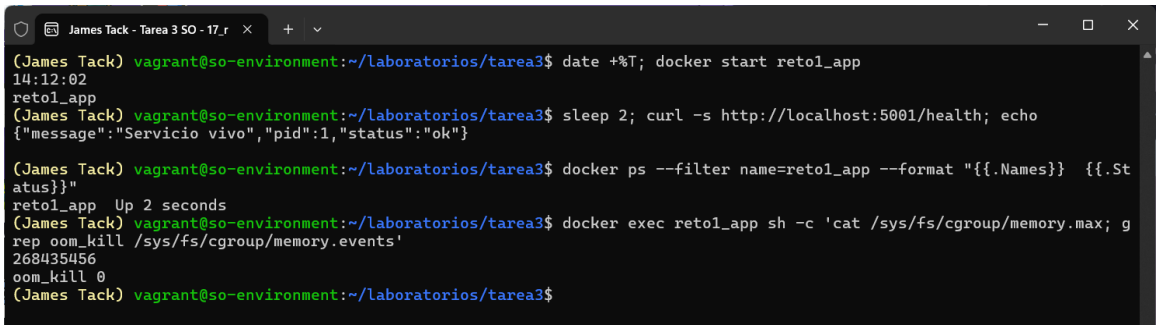


Figura 17. docker start devuelve el servicio con el mismo límite.

Luego repetí la fuga, esta vez con vmstat en otra ventana y leyendo max y oom_kill de memory.events después de cada llamada. El resultado fue idéntico, con las tres primeras bien, la cuarta sin respuesta a las 14:12:36 y otra vez Exited (137). dmesg ya muestra dos Killed process. Por lo tanto, reiniciar no arregla nada, porque la fuga vuelve a llenar el mismo límite en la misma llamada. Los contadores de memory.events siguieron en 0 hasta la tercera llamada porque 231 MB todavía no tocaban el tope. El cambio de la cuarta no alcancé a verlo, porque el cgroup desapareció con el contenedor.

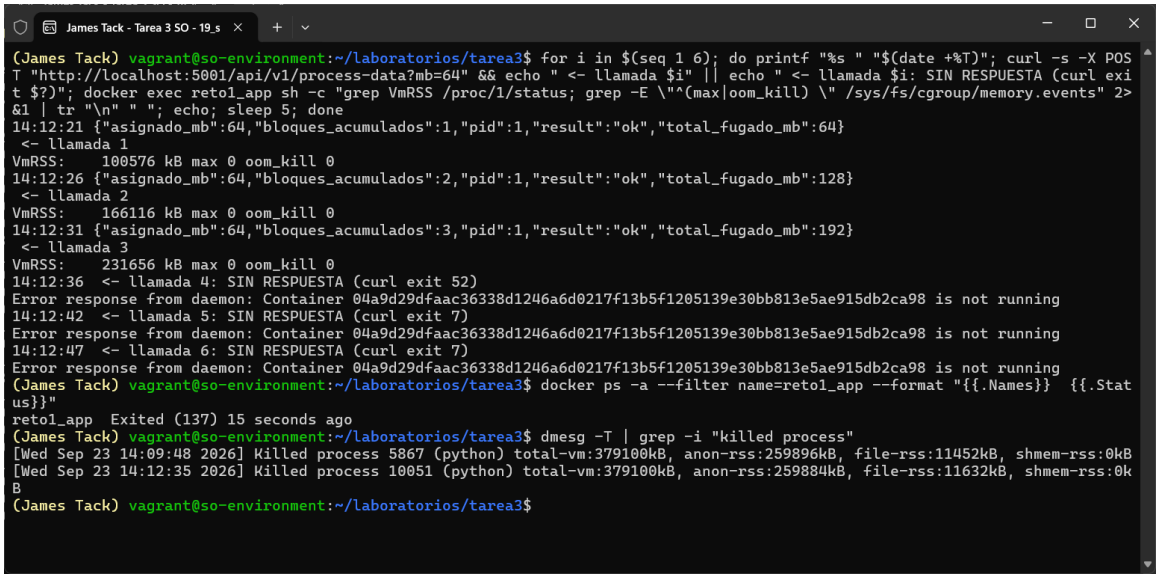


Figura 18. Segunda caída, igual a la primera, y dos entradas Killed process en dmesg.

vmstat muestra que la VM nunca estuvo en apuros. La columna free baja unos 200 MB mientras crece la fuga y los recupera de golpe a las 14:12:38, cuando el kernel mata a python. si y so se quedan en 0 todo el tiempo (la VM no tiene swap) y wa casi no se mueve. Queda más de 1 GB libre

durante todo el incidente. Por eso no es un problema de falta de RAM en el host sino un límite del contenedor más chico que lo que la aplicación termina usando.

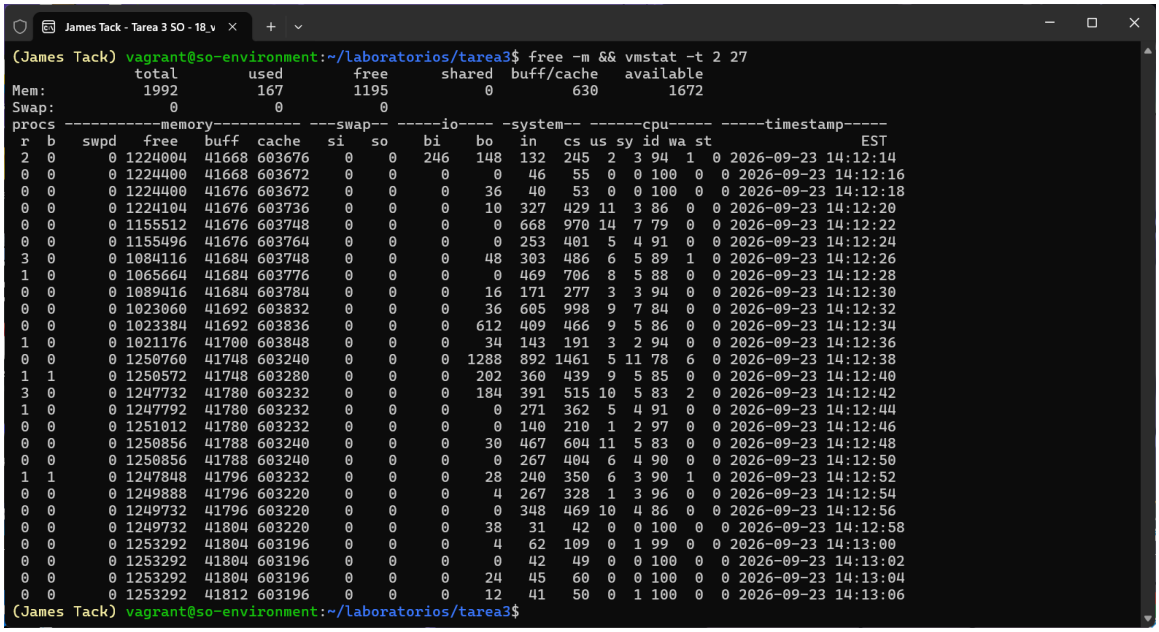


Figura 19. free -m y vmstat -t 2 durante la segunda caída.

Informe RCA

1. Resumen ejecutivo

El servicio de procesamiento de datos se caía porque cada petición deja guardada una porción de memoria que nunca devuelve, y el contenedor tenía un tope de 256 MB. A la cuarta petición grande el sistema operativo lo cerraba a la fuerza para proteger al resto de la máquina. Subí el tope a 512 MB sin reiniciar el servicio y el mismo volumen de trabajo ya no lo tumba, pero la fuga sigue en el código y tiene que corregirla el equipo de desarrollo.

2. Síntoma reportado y síntoma real

Ticket del SOC	Lo que encontré al reproducirlo
Los POST a /process-data responden un rato y luego el servicio deja de responder.	Tres POST de 64 MB responden 200. El cuarto se queda sin respuesta (curl 52) y los siguientes no conectan (curl 7).
El contenedor aparece parado con Exit Code 137.	Exited (137), OOMKilled=true. El kernel registra Killed process (python) por el límite del cgroup.
Vuelve a funcionar al reiniciar y horas después se cae otra vez.	Con docker start vuelve, y se cae en la misma cuarta llamada. Las horas de la queja son solo el tiempo que tarda el tráfico real en acumular unos 230 MB.

3. Timeline

Hora	Evento	Fuente de evidencia

14:08:33	reto1_app arranca con 256 MB y sin swap	docker run, docker events (start)
14:08:44	Contexto verificado. hostname = ID del contenedor y PID 1 = python	docker exec (fig. 8)
14:08:52	Línea base con RSS de 34 MB, oom_kill 0 y memory.max 268435456	/proc/1/status, memory.events (fig. 9 y 10)
14:09:31 a 14:09:43	Llamadas 1 a 3, el RSS sube a 100, 166 y 231 MB	curl, /proc/1/status, docker stats
14:09:48	El kernel mata a python (PID 5867) por el límite del cgroup	dmesg (fig. 13 y 14)
14:09:49	Llamada 4 sin respuesta. Detección del incidente	curl exit 52, docker events oom
14:09:50	El contenedor termina con código 137	docker events die, docker inspect
14:12:03	Mitigación temporal con docker start y la misma configuración	docker start, /health
14:12:35	Segunda muerte por OOM (PID 10051), también en la cuarta llamada	dmesg, docker events, vmstat
14:21:12 a 14:21:16	Fuga de 192 MB liberada con /admin/clear sin reiniciar (231 a 35 MB)	/proc/1/status (fig. 20)
14:21 a 14:22:02	memory.max a 512 MB, primero a mano y luego con docker update	cgroup, docker inspect (fig. 21 y 22)
14:22:28 a 14:22:45	Seis llamadas de 64 MB sin caída. Resolución verificada	curl, memory.events, docker ps (fig. 23)
14:23:24	El límite se mantiene tras docker restart y no hay eventos oom nuevos	memory.max, docker events (fig. 24)

4. Evidencia recopilada

Las salidas están en las figuras 8 a 19, cada una con su comentario. Lo que importa es que cuatro fuentes independientes dicen lo mismo. El kernel (dmesg) registra el OOM del cgroup con uso igual al límite. Docker (inspect, ps y events) marca código 137, OOMKilled=true y un evento oom. El proceso (/proc/1/status) crece 65 MB por llamada hasta quedarse a 25 MB del tope. El cgroup (memory.max, memory.swap.max) muestra 256 MB sin swap. Ninguna de esas fuentes sale de la aplicación, y la aplicación tampoco dejó rastro, que es justo lo que pasa con un SIGKILL.

5. Análisis de causa raíz

Cinco porqués

Pregunta	Respuesta
¿Por qué deja de responder el servicio?	Porque el contenedor termina con código 137 y nadie lo vuelve a levantar (--restart no).
¿Por qué termina con 137?	Porque el kernel le mandó SIGKILL (128 + 9) a python, el PID 1 del contenedor, y con el PID 1 se va el contenedor entero.
¿Por qué el kernel mató a python?	Porque memory.current llegó a memory.max (262144 kB). Con memory.swap.max en 0 no había adónde mover páginas y la memoria anónima no se puede recuperar, así que el OOM Killer del cgroup eligió al único proceso que había.
¿Por qué llegó al límite?	Porque /process-data guarda cada bloque en LEAK_BUCKETS y nunca lo libera. Con 35 MB de base y 64 MB por llamada, la cuarta pide unos 295 MB y el tope es 256.

¿Por qué nadie lo vio antes?	Porque el límite se fijó sin medir el pico real del servicio, no hay alertas sobre memory.events ni sobre el uso del cgroup, y reiniciar a mano hacía desaparecer el síntoma sin investigar la causa.

Clasificación BCP

Categoría	¿Aplica?	Por qué
Bottleneck	Sí, disparador	256 MB sin swap es menos que el pico que alcanza el servicio con su propia fuga.
Complexity	No	Un solo contenedor, un solo proceso y un límite fijo. La cadena causal es corta y se reproduce siempre igual.
People	Sí, origen	El código retiene referencias globales que nunca suelta. Es un defecto de desarrollo y lo tiene que corregir el equipo que lo escribió.
Process	Sí, principal	Se desplegó sin prueba de carga, sin monitoreo de memoria y con una respuesta al incidente que consistía en reiniciar y olvidar.

La causa raíz es la fuga de memoria en /process-data. El límite de 256 MB no la causa, solo determina en qué llamada se nota.

Lo que la hizo recurrente fue la falta de proceso, con nada que midiera la memoria ni avisara antes del OOM.

6. Mitigación aplicada

Si la aplicación sigue viva, la mitigación que no borra evidencia es la ruta administrativa /admin/clear, que vacía LEAK_BUCKETS y fuerza el recolector de basura. La probé con tres llamadas de 64 MB. El RSS estaba en 231564 kB y bajó a 34944 kB, casi el valor de arranque, con el contenedor todavía arriba y oom_kill en 0. Python devuelve esa memoria al sistema enseguida porque bloques de ese tamaño se piden con mmap y se liberan con munmap apenas nadie los referencia.

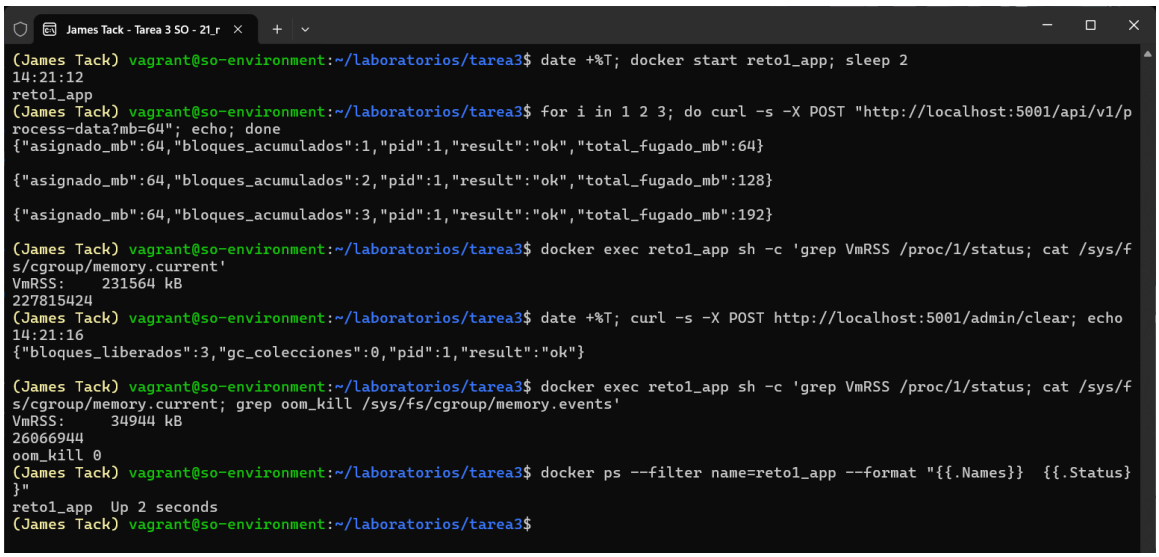


Figura 20. /admin/clear libera los tres bloques y el RSS vuelve a 35 MB sin reiniciar.

En cuanto al impacto, durante el incidente fue total. Entre la muerte del proceso y el reinicio manual, cualquier petición al puerto 5001 fallaba, y las que estaban en curso se perdían sin respuesta. En la prueba el servicio estuvo caído dos minutos y trece segundos (de 14:09:50 a 14:12:03). En producción ese hueco dura lo que tarde alguien en darse cuenta. /admin/clear evita la caída, pero hay que llamarlo antes de que la fuga llegue al tope, así que sirve como parche y no como solución.

7. Solución y prevención

Cambio aplicado

El OOM se dispara porque el pico real no cabe en memory.max. Seis llamadas de 64 MB suman 384 MB de fuga, y con los 35 MB de base y algo de margen para el propio Python y Flask elegí 512 MB. Por eso el swap sigue en cero, porque meter swap aquí solo cambiaría una muerte rápida por un servicio lento que pagina sin parar.

Probé primero la vía de bajo nivel, escribir el límite directamente en el cgroup del contenedor desde el host. Saqué el ID completo con docker inspect y escribí 512M en memory.max con sudo tee. El kernel lo aceptó al instante y desde adentro del contenedor memory.max ya decía 536870912. Sin embargo, docker inspect seguía diciendo 256 MB. Docker no se entera de ese cambio y lo pisaría con su propio valor en cuanto el contenedor se reinicie.

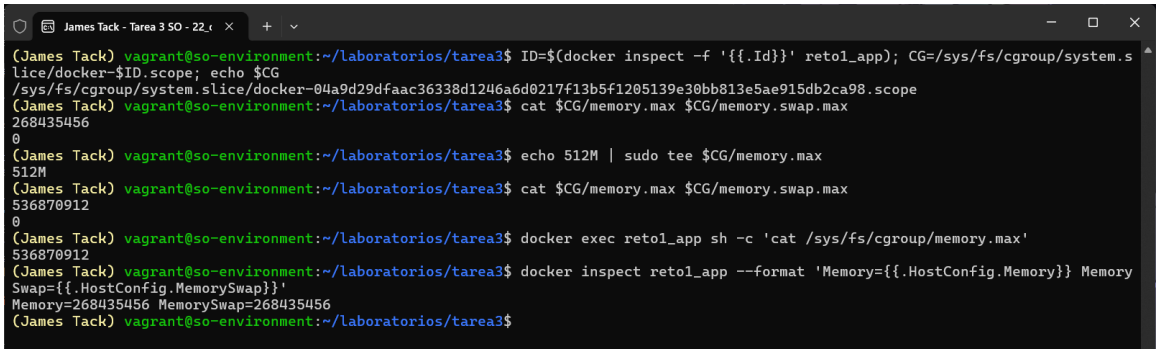


Figura 21. memory.max escrito a mano en el cgroup del host. El kernel cambia y Docker no.

Para que el cambio quede guardado hay que hacerlo a través de Docker, con docker update, que cambia recursos de un contenedor que ya existe sin recrearlo. El primer intento, con solo --memory 512m, lo rechazó con el mensaje Memory limit should be smaller than already set memoryswap limit. Es la regla de coherencia de la guía, --memory-swap no puede quedar por debajo de --memory, y el valor anterior de 256 MB ya lo estaba. Con los dos flags juntos pasó. Docker y el cgroup quedaron de acuerdo en 536870912 bytes, swap en 0, y el contenedor siguió corriendo todo el tiempo.

```
docker update --memory 512m --memory-swap 512m reto1_app
```

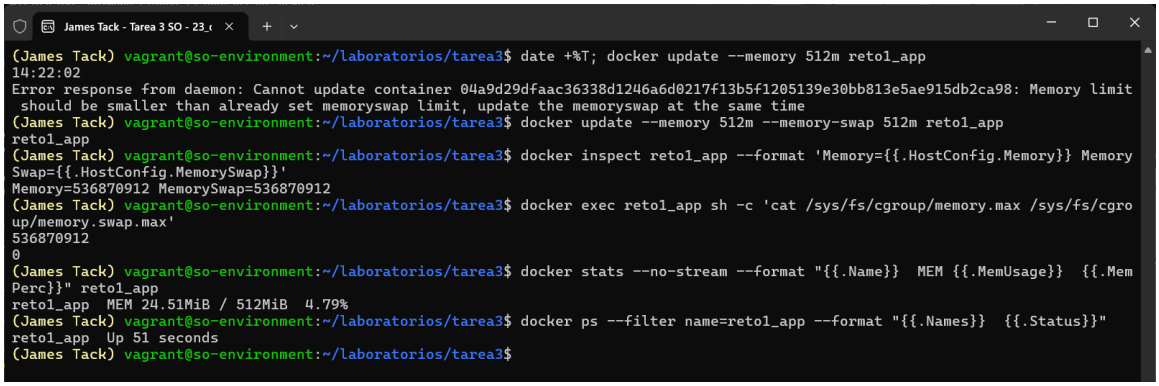


Figura 22. docker update rechaza el cambio incoherente y acepta los dos límites juntos.

Prueba de que ya no se reproduce

Repetí la fuga completa de la guía. Las seis llamadas respondieron, el RSS llegó a 428188 kB con 384 MB fugados, max y oom_kill de memory.events se quedaron en 0 y docker ps siguió en Up. docker stats marcó 409,3 MiB sobre 512 MiB, un 79,95 %, y dmesg sigue teniendo solo las dos muertes de antes.

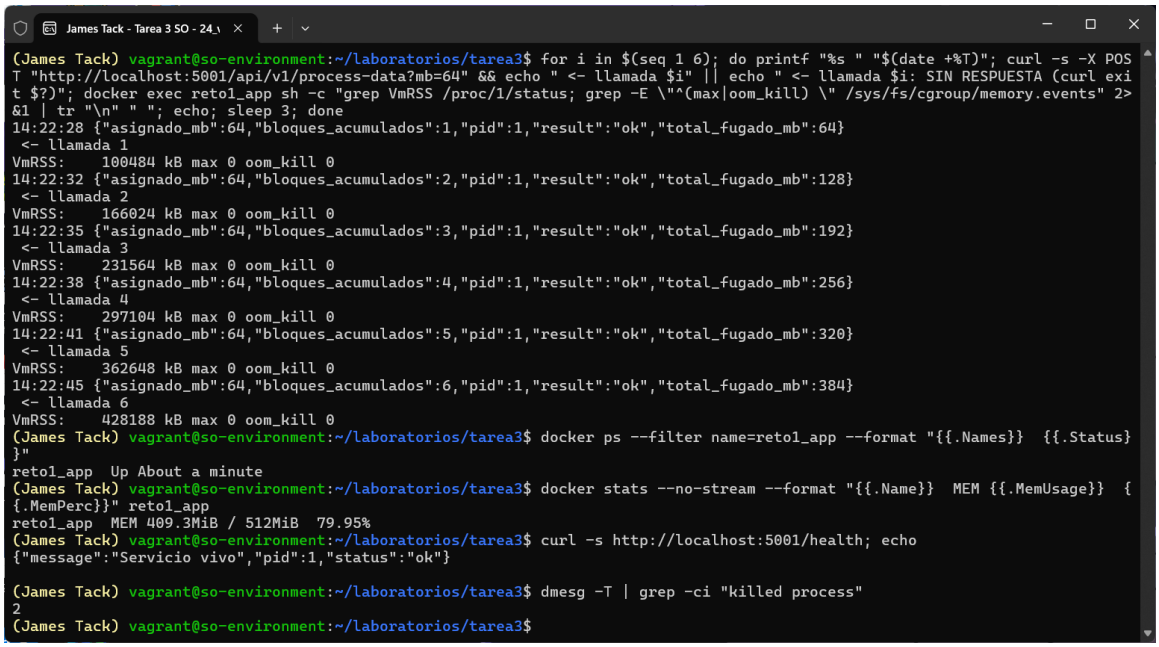


Figura 23. Seis llamadas de 64 MB con el límite nuevo. Sin caída y sin OOM.

Por último liberé la fuga con /admin/clear y reinicié el contenedor con docker restart. memory.max volvió con 536870912, prueba de que el límite de docker update se guardó en la configuración del contenedor y no solo en el cgroup de ese momento. docker events no tiene ningún oom desde las 14:22.

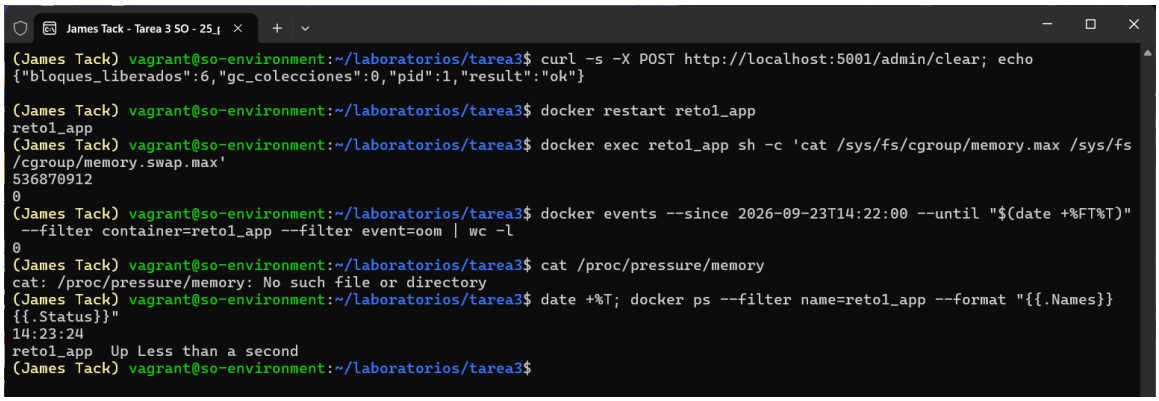


Figura 24. El límite sobrevive a docker restart y no hay eventos oom nuevos. /proc/pressure/memory no existe en este kernel.

Límites y alertas recomendados

- Dimensionar --memory con el pico medido más un margen de alrededor de 25 %. Con la carga de la guía el pico fue de 409 MiB, y 512 MB lo cubre. --memory-swap igual a --memory, sin swap.
- Alertar cuando memory.current pase el 80 % de memory.max y cuando cambie el contador max de memory.events, que avisa antes del OOM. Cualquier incremento de oom_kill, o un evento oom en docker events, tiene que abrir un incidente automáticamente.
- La guía sugiere vigilar /proc/pressure/memory, pero PSI entró en el kernel 4.20 y esta VM tiene 4.15, como se ve en la figura 24. Para usarlo hay que actualizar el kernel. Mientras tanto, memory.events y docker events cubren la misma necesidad.
- Cambiar --restart no por --restart on-failure con un máximo de reintentos, para que el servicio vuelva solo mientras se corrige el código. Eso baja el tiempo de caída pero no reemplaza la alerta, porque un contenedor que se reinicia en silencio esconde el problema, como pasaba con los reinicios manuales.
- Reportar a desarrollo la fuga de LEAK_BUCKETS en /api/v1/process-data. Los datos procesados deberían liberarse al terminar la petición o guardarse fuera del proceso. Mientras no se corrija, cualquier límite solo aplaza la caída. Con 512 MB y bloques de 64 MB, la séptima llamada acumulada todavía cabe (unos 493 MB) y la octava vuelve a pasar el límite.

8. Lecciones aprendidas

La más importante es que el código 137 no es un error de la aplicación sino una muerte por SIGKILL, y por eso docker logs no sirve para explicarlo. La explicación está en dmesg y en el cgroup, y hay que ir a buscarla antes de reiniciar, porque al caer el contenedor se borra su cgroup con todos sus contadores. Si no hubiera guardado dmesg y docker events, lo único que me quedaría del primer incidente sería un Exited (137) sin contexto.

Además aprendí a desconfiar de un límite que se cambia sin que Docker se entere. Escribir en memory.max funciona y sirve para salir del paso en caliente, pero dura hasta el próximo reinicio. docker update hace lo mismo y además lo guarda. Y la regla de --memory-swap, que parecía un detalle, es justo lo que hace fallar el primer docker update.

Al final, subir el límite no arregla una fuga. Solo le da más espacio antes de que vuelva a llenarse. Como SRE mi parte era devolver el servicio y dejar alertas para que la próxima caída no llegue por sorpresa.

La corrección de verdad está en una línea de app.py que no me tocaba cambiar.

Referencias

[1] T. Heo et al., "Control Group v2", The Linux Kernel documentation. [En línea]. Disponible en: https://docs.kernel.org/admin-guide/cgroup-v2.html

[2] "The /proc Filesystem", The Linux Kernel documentation. [En línea]. Disponible en: https://docs.kernel.org/filesystems/proc.html

[3] Docker Inc., "Resource constraints", Docker Docs. [En línea]. Disponible en: https://docs.docker.com/engine/containers/resource_constraints/

[4] Docker Inc., "docker container update", Docker Docs. [En línea]. Disponible en: https://docs.docker.com/reference/cli/docker/container/update/

[5] A. Silberschatz, P. B. Galvin y G. Gagne, Operating System Concepts, 10a ed. Hoboken, NJ, EE. UU.: Wiley, 2018, cap. 10.